

Documentation RedPandaFlow

RedPandaFlow est une application de kanban collaboratif (inspirée de Trello) permettant à des équipes d'organiser leur travail en espaces de travail, tableaux, colonnes et cartes, avec une synchronisation en temps réel entre collaborateurs.

Ce dépôt centralise la documentation transverse du projet : architecture, conteneurisation, pipeline CI/CD et organisation de l'équipe. La documentation propre à chaque service se trouve dans le README de son dépôt.

Sommaire

- [Architecture du projet](#) — schéma, services et communication
- [Conteneurisation](#) — choix Docker, Dockerfiles, réseau, volumes, bonnes pratiques
- [Pipeline CI/CD](#) — GitHub Actions, tests, registre d'images
- [Authentification JWT](#) — fonctionnement de l'authentification
- [Équipe et répartition](#) — membres et organisation

Dépôts

- [redpandaflow-backend](#) — API ASP.NET Core (.NET 10), Clean Architecture, EF Core + PostgreSQL, SignalR
- [redpandaflow-frontend](#) — React 19, Vite, Tailwind CSS, servi par nginx
- [redpandaflow-infra](#) — stack docker-compose pour lancer tout le projet
- [documentation](#) — ce dépôt

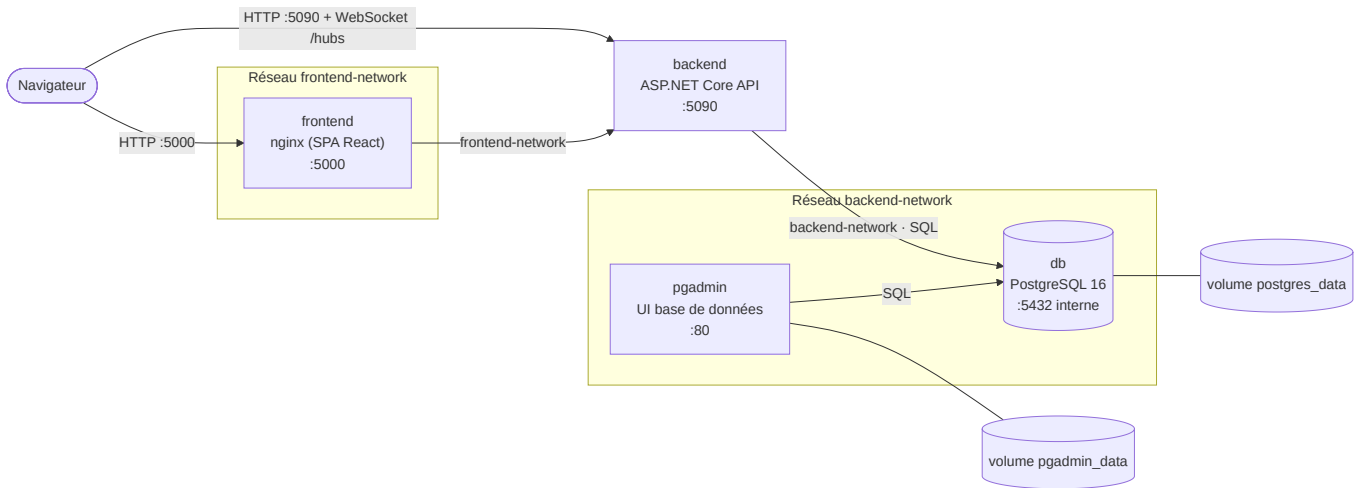
Fonctionnalités principales

- Espaces de travail, tableaux, colonnes et cartes avec glisser-déposer
- Synchronisation temps réel (présence, mutations de tableau, notifications)
- Commentaires, étiquettes, checklists et assignations sur les cartes
- Historique d'activité par carte
- Notifications avec badge de non-lus
- Recherche transverse (espaces, tableaux, cartes)
- Profils utilisateurs avec avatar, suppression de compte
- Rôles sur les espaces et tableaux (Admin, Membre, Lecteur)

Architecture du projet

RedPandaFlow est découpé en quatre services conteneurisés, orchestrés par un unique fichier `docker-compose.yml` (dépôt `redpandaflow-infra`).

Schéma d'architecture



Le `backend` est sur **les deux réseaux** (il fait le pont présentation ↔ données) ; il est représenté hors des cadres pour cette raison.

Services et rôles

Service	Image	Rôle
frontent	<code>nginxinc/nginx-unprivileged</code> (build node)	Sert l'application React (SPA) compilée en statique
backend	<code>.NET 10</code> (build SDK → runtime aspnet)	API REST + hubs SignalR (temps réel)
db	<code>postgres:16-alpine</code>	Base de données relationnelle, persistée sur volume
pgadmin	<code>dpage/pgadmin4</code>	Interface d'administration de la base (service supplémentaire)

Communication entre services

- Le **navigateur** charge la SPA servie par `frontend` (nginx) sur le port `5000` .
- La SPA appelle l'**API** `backend` sur le port `5090` : requêtes REST (authentification, espaces, tableaux, cartes...) et connexions **WebSocket** vers les hubs SignalR (`/hubs/board` , `/hubs/notifications`) pour le temps réel.
- Le `backend` se connecte à la base `db` via le nom de service `db` sur le réseau interne (`Host=db;Port=5432`). La base **n'est pas exposée** à l'hôte.
- `pgadmin` accède à `db` sur le même réseau interne.

Segmentation réseau

Deux réseaux Docker distincts isolent les responsabilités :

- `frontend-network` : relie `frontend` et `backend` .
- `backend-network` : relie `backend` , `db` et `pgadmin` .

Le `backend` est le seul service présent sur les deux réseaux : il fait le pont entre la couche présentation et la couche données. La base de données n'est joignable que sur `backend-network` , jamais depuis le `frontend` ni depuis l'hôte.

Démarrage de la stack

Le `backend` dépend de la santé de la base (`depends_on: condition: service_healthy`) : il n'est lancé qu'une fois que le healthcheck PostgreSQL (`pg_isready`) est au vert, ce qui évite les erreurs de connexion au démarrage.

```
docker compose up -d --build
```

Conteneurisation

Ce document justifie les choix de conteneurisation du projet : rôle de chaque conteneur, contenu des Dockerfiles, organisation du réseau, gestion des volumes et bonnes pratiques appliquées.

Justification des choix

Le projet est conteneurisé dès la phase de développement pour garantir un environnement **identique pour toute l'équipe** et **reproductible** (même versions de runtime, de base de données et de configuration). Tout se lance avec une seule commande `docker compose up`.

L'architecture suit le minimum imposé (API, interface web, base de données) auquel s'ajoute un service supplémentaire, **pgAdmin**, pour administrer la base sans l'exposer publiquement.

Les conteneurs

Conteneur	Image de base	Rôle
backend	build : <code>mcr.microsoft.com/dotnet/sdk:10.0</code> , runtime : <code>mcr.microsoft.com/dotnet/aspnet:10.0</code>	API REST + SignalR
frontend	build : <code>node:26-alpine</code> , runtime : <code>nginxinc/nginx-unprivileged:stable-alpine</code>	SPA React servie par nginx
db	<code>postgres:16-alpine</code>	Base de données
pgadmin	<code>dpage/pgadmin4</code>	Administration de la base

Dockerfiles

Backend (`redpandaflow-backend/Dockerfile`)

Build **multi-étapes** :

1. **Étape `build`** (SDK .NET 10) : restauration des paquets puis `dotnet publish -c Release` de l'API. Les `.csproj` sont copiés et restaurés avant le reste du code pour profiter du cache de couches Docker.
2. **Étape `runtime`** (image `aspnet` , sans le SDK) : on ne copie que le résultat publié → image finale réduite (~264 Mo).

Bonnes pratiques :

- `--no-restore` et `/p:UseAppHost=false` à la publication (build plus léger).
- Exécution en **utilisateur non-root** via `USER $APP_UID` (uid 1654, fourni par l'image .NET).
- Contexte de build **local au dépôt** (chemins `src/ ...`), ce qui permet de construire l'image aussi bien via le compose qu'en CI sur le dépôt seul.

Frontend (`redpandaflow-frontend/Dockerfile`)

Build **multi-étapes** :

1. **Étape builder** (`node:26-alpine`) : `npm ci` (installation reproductible à partir du `package-lock.json`) puis `npm run build` (Vite).
2. **Étape runtime** (`nginx-unprivileged`) : on ne copie que le `dist/` compilé. Image finale ~55 Mo.

Bonnes pratiques :

- Image **nginx non-priviligée** : nginx tourne en utilisateur non-root (uid 101).
- Le port d'écoute de nginx est injecté au démarrage via le mécanisme de templates de l'image (`envsubst` sur `FRONTEND_PORT`), avec un filtre (`NGINX_ENVSUBST_FILTER`) pour ne substituer que cette variable.

Base de données

PostgreSQL utilise directement l'**image officielle** `postgres:16-alpine` : pas de Dockerfile dédié, conformément aux bonnes pratiques (on ne recompile pas une image standard).

Organisation du réseau

Deux réseaux Docker isolent les couches (voir [architecture.md](#)) :

- `frontend-network` : `frontend` ↔ `backend`
- `backend-network` : `backend` ↔ `db` ↔ `pgadmin`

La base de données n'est accessible que sur `backend-network`. Le `backend`, présent sur les deux réseaux, est le seul pont entre présentation et données.

Gestion des volumes et des données

- `postgres_data` : fichiers de la base. Les données survivent à `docker compose down` (seul `down -v` les supprime).

- `pgadmin_data` : configuration de pgAdmin.

Séparation nette entre **données applicatives** (volumes nommés) et **conteneurs** (éphémères, restructurables à tout moment).

Sauvegarde et restauration

Deux scripts dans `redpandaflow-infra/scripts/` opèrent sur le conteneur `db` en cours d'exécution (ils lisent les identifiants directement dans le conteneur) :

```
./scripts/backup.sh                # dump horodaté dans backups/  
./scripts/restore.sh backups/<fichier>.dump # restauration
```

Le format `pg_dump -Fc` (custom) permet une restauration avec `pg_restore --clean --if-exists`.

Pipeline CI/CD

Choix de la plateforme

Le projet est hébergé sur une **organisation GitHub** (`RedPandaFlow`). Le pipeline est donc réalisé avec **GitHub Actions** plutôt que GitLab CI : nous n'avons pas d'accès à une instance GitLab ni à la plateforme Scalingo dans le cadre de ce projet. Les principes restent identiques (build, tests, publication d'image dans un registre) ; seuls les outils changent.

Pour la même raison, **le déploiement automatisé n'est pas mis en place** : il nécessiterait un accès à Scalingo (ou une autre plateforme) et les identifiants associés. Le reste de la chaîne (intégration continue + publication d'image) est en place.

Pipeline backend

Fichier : `redpandaflow-backend/.github/workflows/ci.yml` .

Déclenchement

- `push` sur `dev` et `main`
- `pull_request` vers `dev` et `main`

Un mécanisme de `concurrency` annule un run précédent encore en cours sur la même branche.

Étapes (jobs)

Job	Rôle
Secret scan	Scan de l'historique git avec <code>gitleaks</code> ; échoue si un secret est détecté
Build (.NET)	<code>dotnet restore</code> + <code>dotnet build -c Release</code> de l'API
Test (.NET)	<code>dotnet test</code> du projet xUnit (<code>tests/RedPandaFlow.Tests</code>)
Build & push image	Construit l'image Docker et la publie sur le registre

Le job d'image dépend de la réussite de `Build` et `Test` (`needs: [build, test]`).

Tests automatisés

Le projet `RedPandaFlow.Tests` (xUnit) couvre de la logique pure, sans base de données :

- `ServiceResult<T>` : fabrique des résultats `Ok` / `Fail` .
- `JwtTokenService` : génération du jeton d'accès (claims utilisateur), du jeton de rafraîchissement, et relecture d'un jeton.

Registre d'images

Les images sont publiées sur le **GitHub Container Registry (ghcr.io)**, équivalent GitHub du GitLab Container Registry :

```
ghcr.io/redpandaflow/redpandaflow-backend
```

Tags générés automatiquement (via `docker/metadata-action`) :

- nom de la branche (ex. `dev`)
- SHA du commit
- `latest` (uniquement sur `main`)

La publication se fait sur un `push` ou sur une pull request après la **construction** (validation) de l'image vers `dev` / `main` . Le job d'image est configuré pour ne pas s'exécuter sur les autres branches.

Sécurité du pipeline

- `permissions: contents: read` au niveau du workflow (principe du moindre privilège) ; seul le job d'image obtient `packages: write` .
- Authentification au registre via le `GITHUB_TOKEN` intégré (aucun secret personnel stocké).
- Aucun secret ni port sensible en clair dans le workflow ; scan `gitleaks` pour empêcher toute fuite.

Pipeline frontend

Fichier : `redpandaflow-frontend/.github/workflows/ci.yml` .

Déclenchement

- `push` sur `dev` et `main`
- `pull_request` vers `dev` et `main`

Un mécanisme de `concurrency` annule un run précédent encore en cours sur la même branche.

Étapes (jobs)

Job	Rôle
Secret scan	Scan de l'historique git avec <code>gitleaks</code> ; échoue si un secret est détecté
Build & Lint	Construit l'application et exécute le linting pour faire les tests
Docker Build	Construit l'image Docker et la publie sur le registre

Le job d'image dépend de la réussite de `Secret scan` **et** `Build & Lint` (`needs: [secret-scan, build-and-lint]`).

Tests automatisés

Les tests sont réalisés via le linting (ESLint) : il vérifie la syntaxe, les règles de style, et peut aussi détecter des erreurs potentielles (ex. variable non utilisée). Il n'y a pas de tests unitaires formels, mais le linting assure une validation de base du code.

Registre d'images

Les images sont publiées sur le **GitHub Container Registry (ghcr.io)**, équivalent GitHub du GitLab Container Registry :

```
ghcr.io/redpandaflow/redpandaflow-frontend
```

Tags générés automatiquement (via `docker/metadata-action`) :

- nom de la branche (ex. `dev`)
- SHA du commit
- `latest` (uniquement sur `main`)

La publication se fait sur un `push` ou sur une pull request après la **construction** (validation) de l'image vers `dev / main` . Le job d'image est configuré pour ne pas s'exécuter sur les autres branches.

Sécurité du pipeline

- `permissions: contents: read` au niveau du workflow (principe du moindre privilège) ; seul le job d'image obtient `packages: write` .
- Authentification au registre via le `GITHUB_TOKEN` intégré (aucun secret personnel stocké).
- Aucun secret ni port sensible en clair dans le workflow ; scan `gitleaks` pour empêcher toute fuite.

Protection des branches

Des *rulesets* GitHub protègent `main` et `dev` : pull request obligatoire, blocage du force-push, et passage obligatoire des checks de la CI avant fusion.

Authentification JWT

Ce document explique comment fonctionne l'authentification dans RedPandaFlow.

Le principe

Quand un utilisateur se connecte, le serveur va lui remettre deux passes numériques (des "jetons"). À chaque action de l'utilisateur, son navigateur présente ces jetons et le serveur vérifie qu'ils sont valides avant d'autoriser l'action.

Aucun mot de passe ne circule entre le navigateur et le serveur après la connexion. Seuls les jetons sont échangés.

Les deux jetons

L'application utilise deux jetons aux rôles différents.

Le jeton d'accès est utilisé pour prouver l'identité à chaque requête. Il a une durée de vie courte (quinze minutes). S'il est volé, le voleur n'a qu'une fenêtre très limitée pour s'en servir.

Le jeton de rafraîchissement sert uniquement à obtenir un nouveau jeton d'accès quand l'ancien expire. Il a une durée de vie plus longue (sept jours), mais il n'est utilisable que sur un seul endpoint dédié.

Ce système permet à l'utilisateur de rester connecté plusieurs jours sans ressaisir son mot de passe, tout en limitant les dégâts en cas de vol.

Où sont stockés les jetons

Les deux jetons sont placés dans des cookies dits "HttpOnly". Cela signifie simplement que le code JavaScript de la page n'a pas le droit de les lire. Seul le navigateur sait qu'ils existent et les renvoie automatiquement au serveur à chaque requête.

Cela protège contre une attaques très répandues (injection de script malveillant dans la page) qui chercherait à voler les jetons.

L'application ne stocke jamais les jetons dans le stockage local du navigateur, ni dans une variable JavaScript.

Le cycle de vie d'une session

1. L'utilisateur s'inscrit ou se connecte avec son email et son mot de passe. Le serveur vérifie les identifiants et dépose les deux cookies dans la réponse.
2. À chaque page visitée ou action effectuée, le navigateur joint automatiquement le jeton d'accès. Le serveur vérifie sa signature et autorise l'action.
3. Quand le jeton d'accès expire (au bout de quinze minutes), la requête suivante échoue avec un code d'erreur. Le navigateur le détecte, appelle automatiquement l'endpoint de rafraîchissement, obtient un nouveau jeton d'accès et un nouveau jeton de rafraîchissement, puis rejoue la requête initiale. L'utilisateur ne voit rien de tout cela.
4. Quand l'utilisateur clique sur "Se déconnecter", le serveur invalide les jetons et vide les cookies. Une nouvelle connexion sera nécessaire.

Protections supplémentaires

Rotation des jetons de rafraîchissement. À chaque rafraîchissement, le jeton précédent est marqué comme utilisé et remplacé par un nouveau. Un même jeton de rafraîchissement ne peut servir qu'une seule fois.

Détection de réutilisation. Si un ancien jeton de rafraîchissement est présenté une seconde fois (signe qu'il a été volé et que deux personnes essaient de s'en servir), tous les jetons actifs de l'utilisateur sont révoqués. La session est alors fermée partout, par sécurité.

Hachage des mots de passe. Le mot de passe n'est jamais stocké en clair dans la base de données. Seule une empreinte calculée avec l'algorithme bcrypt est conservée. Même si la base de données venait à fuiter, les mots de passe resteraient protégés.

Limitation du nombre de tentatives. Les endpoints de connexion, d'inscription et de rafraîchissement sont limités à cinq requêtes par minute et par adresse IP, pour décourager les tentatives automatisées.

Liste des endpoints concernés

Endpoint	Rôle
POST /api/auth/register	Créer un nouveau compte
POST /api/auth/login	Se connecter avec email + mot de passe
POST /api/auth/refresh	Obtenir un nouveau jeton d'accès
GET /api/auth/me	Récupérer le profil de l'utilisateur connecté
POST /api/auth/logout	Se déconnecter et invalider les jetons
DELETE /api/auth/account	Supprimer définitivement son compte

Équipe et répartition des tâches

Le projet est réalisé par un binôme.

Membres

- Nathan FERRE
- Ylan Dessenne

Répartition

Le travail a été mené de façon **collaborative** : les deux membres ont contribué à l'ensemble des parties du projet (backend, frontend, infrastructure Docker, CI/CD et documentation), en se répartissant les tâches au fil du développement plutôt que par domaine fixe.

Domaine	Contributeurs
API backend (.NET)	Nathan FERRE, Ylan Dessenne
Frontend (React)	Nathan FERRE, Ylan Dessenne
Conteneurisation & infra Docker	Nathan FERRE, Ylan Dessenne
CI/CD (GitHub Actions)	Nathan FERRE, Ylan Dessenne
Documentation	Nathan FERRE, Ylan Dessenne